Reviewer Pack — Minimal Rank of Quasi-Postnikov Sections and Monotone Circui…

Entry 3645c4b4bb4d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 06:47:49 UTC

Minimal Rank of Quasi-Postnikov Sections and Monotone Circuit Depth

Entry ID: `3645c4b4bb4d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 06:47:49 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology: Quasi-Postnikov Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

> ['For any given CNF formula, the minimal rank of a quasi-Postnikov section associated with its resolution graph is upper-bounded by the depth of its corresponding monotone circuit.', 'This bound is tight if and only if the CNF formula is in P.', 'Equivalently, for every instance n with property P, there exists a monotone circuit of depth O(n^2) that correctly computes the truth table of the CNF.']

Rationale (proposer's reasoning):

> ['Quasi-Postnikov sections provide a way to study the homotopy type of spaces by simplifying their descriptions. Their ranks could potentially encode structural information about the complexity of computing properties of these spaces.', 'The tight bound implies that the resolution graph of a CNF has a simple structure, which is consistent with known results about monotone circuits.']

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ba5c024273c8ab34`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all CNF formulas n ≤ 40, the minimal rank of a quasi-Postnikov section is less than or equal to the depth of its monotone circuit and the difference between ranks and depths for at least 80% of seeds is ≤ 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quasi-Postnikov sections" AND "monotone circuit depth" - "minimal rank" AND "CNF formula" AND "upper bound" - "truth table computation" AND "depth O(n^2)" AND "monotone circuit"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if sum(clause) != 0:
                clauses.append(clause)
        return clauses

    def resolution_graph(cnf):
        graph = {}
        for i in range(len(cnf)):
            for j in range(i + 1, len(cnf)):
                if any(abs(x) == abs(y) and x * y < 0 for x in cnf[i] for y in cnf[j]):
                    for var in set(cnf[i]) | set(cnf[j]):
                        if var not in graph:
                            graph[var] = []
                        if -var not in graph:
                            graph[-var] = []
                        if j not in graph[var]:
                            graph[var].append(j)
                        if i not in graph[-var]:
                            graph[-var].append(i)
```

```
    return graph

def minimal_rank(graph):
    rank = 0
    visited = set()
    for node in graph:
        if node not in visited:
            queue = [node]
            while queue:
                current = queue.pop(0)
                if current not in visited:
                    visited.add(current)
                    for neighbor in graph[current]:
                        if neighbor not in visited:
                            queue.append(neighbor)
            rank += 1
    return rank

def monotone_circuit_depth(cnf):
    n = len(cnf)
    graph = resolution_graph(cnf)

    def dfs(node, parent):
        nonlocal depth
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                if neighbor != parent:
                    dfs(neighbor, node)
            depth = max(depth, len(visited))

    visited = set()
    depth = 0
    for i in range(n):
        dfs(i, None)
    return depth

n = random.randint(5, 40)
cnf = generate_cnf(n)
graph = resolution_graph(cnf)
rank = minimal_rank(graph)
depth = monotone_circuit_depth(cnf)

if rank > depth:
    return {
        "metric_name": "Rank vs Depth",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"CNF with n={n} has rank {rank} and depth {depth}"
    }

return {
    "metric_name": "Rank vs Depth",
    "metric_value": abs(rank - depth),
    "instances_tested": 1,
```

```
        "conjecture_holds": abs(rank - depth) <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 100))[:30]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = min((result["counterexample"] for result in results if not result["conjec
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e24d298.py", line 108, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e24d298.py", line 82, in run_trial
    rank = minimal_rank(graph)
           ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e24d298.py", line 54, in minimal_rank
    for neighbor in graph[current]:
                    ~~~~~^^^^^^^^^
KeyError: 0
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate the cause of the crash and rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14290
2	preregistration	ollama_remote	glm4:latest	0	0	9624
3	novelty	ollama_remote	glm4:latest	0	0	8741
4	novelty	ollama_remote	glm4:latest	0	0	8833
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14561
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13169
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10163
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10135
9	judge	ollama_remote	glm4:latest	0	0	11832

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 101348 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3645c4b4bb4d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3645c4b4bb4d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3645c4b4bb4d.tar.gz` (if generated)